

Day 3: OOP Fundamentals (11-June-2026, Thursday)

Session Time: 9:00 AM – 4:30 PM (Lunch Break: 12:30 PM – 1:30 PM, Tea Breaks: 10:30 AM & 3:00 PM)

Session Objectives

By the end of Day 3, learners will be able to:

- Define classes and instantiate objects using constructors.
- Differentiate between instance variables and local variables.
- Use the `this` keyword to refer to current object and resolve naming conflicts.
- Apply the `static` keyword to variables, methods, and blocks.
- Implement encapsulation using private fields and public getters/setters.
- Solve recursive problems like Climbing Stairs.
- Merge two sorted arrays efficiently.
- Understand HackerRank problems: Java Methods, Java Static Initializer Block.
- Design a component of an **HR Management Portal** (employee records, role-based access).

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 2 & Day 3 Overview	15 min
9:15 – 10:00	Classes & Objects – Blueprint and Instances	45 min
10:00 – 10:45	Constructors – Default, Parameterized, Overloading	45 min
10:45 – 11:00	Tea Break	15 min
11:00 – 11:30	<code>this</code> Keyword – Referring to current object	30 min
11:30 – 12:15	<code>static</code> Keyword – Variables, Methods, Blocks	45 min
12:15 – 12:30	Hands-on: Static vs Instance Counter Exercise	15 min
12:30 – 1:30	Lunch Break	60 min
1:30 – 2:15	Encapsulation – Private + Getters/Setters	45 min

2:15 – 2:45	HackerRank: Java Methods & Static Initializer Block	30 min
2:45 – 3:00	Tea Break	15 min
3:00 – 3:30	LeetCode: Climbing Stairs (Recursion & DP intro)	30 min
3:30 – 4:00	LeetCode: Merge Sorted Array (Two-pointer technique)	30 min
4:00 – 4:30	Corporate Use Case: HR Management Portal	30 min

1. Classes & Objects

1.1 Object-Oriented Programming (OOP) Paradigm

- OOP organizes code around **objects** (data + behavior) rather than functions and logic.
- Benefits: modularity, reusability, data hiding, easier maintenance.
- Four pillars: **Encapsulation, Inheritance, Polymorphism, Abstraction** (Day 4 covers others).

1.2 Class – Blueprint

- A class is a template or blueprint that describes the state (fields) and behavior (methods) of objects.
- **Syntax:**

```
class ClassName {  
    // fields (instance variables)  
    // constructors  
    // methods  
}
```

1.3 Object – Instance

- An object is a concrete occurrence of a class, created using the `new` keyword.
- Memory allocated in heap.
- Each object has its own copy of instance variables.

1.4 Example Conceptual

- Class: `Employee` – has fields `name`, `id`, `salary` and methods `work()`, `takeBreak()`.
- Objects: `emp1`, `emp2` – each with their own name, id, salary.

1.5 Instance Variables vs Local Variables

Feature	Instance Variable	Local Variable
---------	-------------------	----------------

Feature	Instance variable	Local variable
Declaration location	Inside class, outside any method/block	Inside method, constructor, or block
Default value	Yes (0, false, null)	No – must initialize before use
Scope	Entire class (all non-static methods)	Within the block/method only
Memory allocation	Heap (as part of object)	Stack
Access modifiers	Allowed	Not allowed

1.6 Creating and Using Objects

```
// Declaration + instantiation
Employee emp = new Employee();

// Accessing fields (if public – but we prefer encapsulation)
emp.name = "John";

// Calling methods
emp.work();
```

2. Constructors

2.1 Definition

- A special method invoked automatically when an object is created.
- Same name as the class, no return type (not even `void`).
- Purpose: initialize the object's state.

2.2 Default Constructor

- If no constructor is defined, Java provides a **no-argument default constructor** that does nothing.
- If any constructor is defined, the default constructor is **not** provided.

2.3 Parameterized Constructor

- Allows passing initial values at the time of object creation.
- Avoids separate setter calls.

2.4 Constructor Overloading

- Multiple constructors in the same class with different parameter lists.
- Enables flexible object creation.

2.5 Constructor Chaining

- `this()` – calls another constructor of the same class (must be first statement).
- `super()` – calls parent constructor (covered Day 4).

2.6 Important Notes

- Constructors can have access modifiers (public, private, etc.).
- Private constructor used in Singleton pattern (Day 11).
- Constructor cannot be `static`, `final`, or `abstract`.

3. `this` Keyword

3.1 What is `this`?

- A reference to the **current object** (the object whose method or constructor is being called).
- Used inside instance methods and constructors.
- Cannot be used in `static` contexts (because no current object).

3.2 Common Use Cases

Use Case	Example
Distinguish instance variables from parameters (naming conflict)	<code>this.name = name;</code>
Call another constructor of the same class	<code>this(parameter);</code> (must be first line)
Return the current object from a method (method chaining)	<code>return this;</code>
Pass current object as argument to another method	<code>someMethod(this);</code>

3.3 Why is `this` Needed?

- Without `this`, shadowed variables would cause confusion.
- Example:

```
class Person {
    String name;
    void setName(String name) {
        name = name; // assigns parameter to itself, not instance variable
    }
}
```

Correct: `this.name = name;`

3.4 `this` with Method Calls

- When you call an instance method from another instance method, Java implicitly uses `this`.
- `this.method()` is optional; `method()` works the same.

4. `static` Keyword

4.1 Overview

- `static` members belong to the **class itself**, not to any particular instance.
- Shared across all objects of that class.
- Memory allocated once in the **method area** (not per object).

4.2 Static Variables (Class Variables)

- Used for constants, counters, or shared resources.
- Accessed via `ClassName.variableName` (or via object reference, but not recommended).
- Example: `public static int employeeCount;`

4.3 Static Methods

- Can be called without creating an object: `ClassName.methodName()`.
- Restrictions:
 - Cannot directly access instance variables or methods (no `this` reference).
 - Can access static variables and static methods.
- Common examples: `Math.sqrt()`, `Arrays.sort()`, `System.out`.

4.4 Static Blocks (Static Initializers)

- Executed once when the class is first loaded into memory (before any object creation or static method call).
- Used to initialize static variables with complex logic.
- Multiple static blocks run in order of appearance.

4.5 Static Nested Classes (covered later)

- A class defined inside another with `static` – can be instantiated without outer class instance.

4.6 When to Use Static?

Scenario	Use Static?
Utility method (e.g., <code>Math.max</code>)	Yes
Constant (e.g., <code>PI</code>)	Yes (with <code>final</code>)
Counter tracking number of objects	Yes
Method that depends on instance state	No
Overriding behavior (polymorphism)	No – static methods are not overridden

4.7 Common Pitfalls

- Static method trying to call instance method → compile error.
 - `main()` is static because JVM calls it before any object exists.
 - Overloading static methods is allowed; overriding is not.
-

5. Encapsulation

5.1 Definition

- Wrapping data (fields) and code (methods) into a single unit, and **hiding internal details**.
- Achieved by:
 - Declaring fields as `private` .
 - Providing `public` getter and setter methods to access and modify fields.

5.2 Why Encapsulation?

- **Control** – you can add validation in setters (e.g., age cannot be negative).
- **Flexibility** – internal representation can change without affecting external code.
- **Security** – prevents unauthorized access.
- **Maintainability** – changes limited to one class.

5.3 Getters and Setters

- Naming convention: `getFieldName()` for retrieval, `setFieldName(value)` for modification.
- For boolean fields, `isFieldName()` is common instead of `get`.
- Can choose to provide only getter (read-only) or only setter (write-only).

5.4 Encapsulation vs Data Hiding

- Encapsulation is a broader concept; data hiding (via `private`) is one mechanism.

5.5 Example (Conceptual)

- Class `BankAccount` has private field `balance`.
- Public method `deposit(amount)` validates `amount > 0`, then updates balance.
- Public method `getBalance()` returns balance but does not allow direct modification.

5.6 JavaBeans Standard

- A JavaBean is a class that follows:
 - Public no-argument constructor.
 - Private fields.
 - Public getters/setters.
 - Serializable (optional).

6. LeetCode Problems – Approach Discussion

6.1 LeetCode: Climbing Stairs

- **Problem:** You are climbing a staircase with n steps. Each time you can take 1 or 2 steps. How many distinct ways to reach the top?
- **Pattern:** Dynamic programming / Recursion.
- **Recurrence relation:** $ways(n) = ways(n-1) + ways(n-2)$ with base cases: $ways(0)=1$, $ways(1)=1$.
- **Approaches:**
 - Simple recursion – $O(2^n)$ exponential, inefficient.
 - Memoization (top-down DP) – $O(n)$.
 - Iterative (bottom-up DP) – $O(n)$ time, $O(1)$ space (use two variables).
- **Connection to OOP:** Not direct, but recursive methods are methods that call themselves.
- **Edge cases:** $n=0$ (maybe 1 way), $n=1$, $n=2$.

6.2 LeetCode: Merge Sorted Array

- **Problem:** You are given two sorted arrays `nums1` (size $m+n$) and `nums2` (size n). Merge

nums2 into nums1 such that nums1 becomes sorted.

- **Key insight:** Merge from the back to avoid overwriting unprocessed elements.
 - **Algorithm (two-pointer from end):**
 - Initialize $p1 = m-1$ (last valid element in nums1), $p2 = n-1$, $p = m+n-1$.
 - While $p2 \geq 0$:
 - If $p1 \geq 0$ and $nums1[p1] > nums2[p2]$, set $nums1[p--] = nums1[p1--]$
 - Else set $nums1[p--] = nums2[p2--]$
 - **Time complexity:** $O(m+n)$, space $O(1)$.
 - **OOP relevance:** This logic can be encapsulated in a utility class with static method.
-

7. HackerRank Problems – Approach Discussion

7.1 Java Methods

- **Task:** Create a method that takes parameters and returns a value; often involves arithmetic or logic.
- **Skills practiced:** Method declaration, return types, parameter passing, calling methods from `main`.
- **Example concept:** Write a method `findMax(int a, int b, int c)` that returns the largest.

7.2 Java Static Initializer Block

- **Task:** A class has two static variables B and H. The static block checks if $B > 0$ and $H > 0$; if yes, $area = B * H$; else print exception message.
 - **Purpose:** Demonstrate static block usage for validation before object creation.
 - **Key learning:** Static blocks run at class loading; they can handle complex initialization and throw exceptions.
-

8. Corporate Use Case: HR Management Portal

8.1 Problem Context

An organization needs an HR portal to manage employee records. Requirements:

- Store employee details (name, id, department, salary, joining date).
- Ensure data integrity (salary > 0 , id unique, name not empty).
- Track total number of employees (company-wide).
- Provide read-only access to employee id (cannot be changed after creation).
- Allow salary updates with approval validation (e.g., increase $\leq 20\%$ without manager

approval).

- Generate employee reports.

8.2 Application of Day 3 Topics

Requirement	Java Feature Used
Employee as real-world entity	Class <code>Employee</code>
Store multiple employee attributes	Instance variables (private for encapsulation)
Prevent invalid salary/name	Encapsulation with setter validation
Employee ID immutable after creation	Setter not provided; value assigned via constructor
Count number of employees created	<code>static int employeeCount</code> (incremented in constructor)
Initialize static counter	Static block (or inline initialization)
<code>this</code> to resolve parameter-field name clash	<code>this.name = name;</code> in constructor
Provide multiple ways to create employee	Constructor overloading (e.g., only name, or full details)
Getter methods for report generation	<code>public getName()</code> , <code>getId()</code> , etc.
Method to calculate annual bonus (based on salary)	Instance method <code>calculateBonus()</code>
Utility method to compare two employees by salary	<code>static method compareSalary(Employee e1, Employee e2)</code>
Validate salary increase (cannot exceed 20% without flag)	Setter with business logic and boolean flag parameter

8.3 Example Class Blueprint (Conceptual)

```
public class Employee {
    private static int totalEmployees = 0;
    private static final double MAX_SALARY_INCREASE_RATIO = 0.20;

    private final int empId;           // immutable after construction
    private String name;
    private double salary;
    private String department;

    static {
        System.out.println("Employee class loaded. System ready.");
    }
}
```

```

        // Increment employee count and update system ready flag
        totalEmployees = 0;
    }

    public Employee(String name, double salary, String department) {
        this.name = name;
        this.salary = salary;
        this.department = department;
        this.empId = ++totalEmployees; // auto-generate ID
    }

    // Overloaded constructor: only name and department
    public Employee(String name, String department) {
        this(name, 30000.0, department); // this() constructor chaining
    }

    // Getter for empId (no setter)
    public int getEmpId() { return empId; }

    // Encapsulated salary setter with validation
    public void setSalary(double newSalary, boolean isManagerApproved) {
        double increaseRatio = (newSalary - salary) / salary;
        if (increaseRatio <= MAX_SALARY_INCREASE_RATIO || isManagerApproved) {
            this.salary = newSalary;
        } else {
            System.out.println("Increase exceeds 20% without approval.");
        }
    }

    public double getSalary() { return salary; }

    // Static method to get total employees
    public static int getTotalEmployees() {
        return totalEmployees;
    }
}

```

8.4 Discussion Questions

- Why should `empId` be declared `final` ? (Because it should never change after object creation.)
- How would you ensure no two employees have the same ID? (Static counter ensures uniqueness.)
- What if we need to reset the counter for testing? (Provide a static reset method, but use cautiously.)
- How would you prevent creation of an employee with negative salary? (Validation inside constructor and setters.)
- Why is `MAX_SALARY_INCREASE_RATIO` static and final? (Constant shared across all employees.)

9. Hands-on Exercises (Without Code Lines)

9.1 Class Diagram to English

Given a class diagram with fields `-studentId: int`, `-name: String`, `-gpa: double`, and methods `+getName(): String`, `+setGpa(double): void`, `+calculateRank(): String`.

Describe in words:

- What does the minus sign mean? (private)
- What does the plus sign mean? (public)
- Which field is likely read-only? Why? (studentId – no setter shown)

9.2 Constructor Overloading Scenarios

For class `Book` with fields `title`, `author`, `price`, suggest:

- A default constructor (what default values would you assign?)
- A constructor that takes only title and author (set price to a default like 9.99).
- A constructor with all three fields.
- Why would you need overloaded constructors?

9.3 Static vs Instance – Predict Behavior

Assume a class with:

- `static int x = 0;`
- `int y = 0;`
- `void increment() { x++; y++; }`
- `static void printX() { System.out.println(x); }`

If we create two objects `a` and `b`, call `a.increment(); b.increment();` then print `a.y` and `b.y` and `x`. What are the outputs?

9.4 Encapsulation Validation

Describe a setter method for a field `age` that:

- Only allows values between 18 and 65.
- If value is invalid, sets age to a default of 25 and logs an error message.

9.5 Tracing Recursion (Climbing Stairs)

Trace manually the recursive calls for `climbStairs(3)` using the recurrence `ways(n)=ways(n-1)+ways(n-2)`. How many calls to `ways(1)` are made?

10. Quiz & Quick Check (Conceptual)

1. Which of the following is true about the default constructor?

- (a) It is always provided even if you define a parameterized constructor.
- (b) It initializes instance variables to their default values.
- (c) It cannot be overridden.
- (d) It can be private.

2. What does the `this` keyword refer to?

- (a) The superclass object.
- (b) The current class's static context.
- (c) The current object.
- (d) The method parameter.

3. Which of these can be declared static?

- (a) Instance variable.
- (b) Constructor.
- (c) Local variable inside a method.
- (d) Nested class.

4. Encapsulation is achieved by:

- (a) Declaring all methods as private.
- (b) Declaring all fields as public and methods as private.
- (c) Declaring fields as private and providing public getters/setters.
- (d) Using inheritance.

5. Given `static int count = 0;` in a class, and two objects are created, what is `count` after both creations if constructor contains `count++` ?

- (a) 0
- (b) 1
- (c) 2
- (d) Compilation error

Answers: 1-b, 2-c, 3-d (static nested class), 4-c, 5-c

11. Group Activity – Pair Programming Simulation

- **Duration:** 20 minutes (during hands-on slot).
 - **Task:** Design (pseudocode/class skeleton) an **Employee** class for HR portal with:
 - Private fields: id (auto-generated static counter), name, salary, department.
 - Constructor that takes name, salary, department.
 - Getter for id, getters/setters for others with validation: salary must be $\geq$ minimum wage (use a static final constant).
 - Static method to get total employee count.
 - Instance method `giveRaise(percent)` that increases salary by percent if positive.
 - **Outcome:** Pairs present their design; instructor highlights proper use of `static`, `this`, encapsulation.
-

12. Additional Resources & Cheat Sheets

- **Constructor rules:** Same name as class, no return type, can overload, can be chained using `this()`.
- **static cheat sheet:**
 - Variables → class level, one copy.
 - Methods → cannot access instance members.
 - Blocks → run on class loading.
 - Import static – allows static members to be used without class name.
- **Encapsulation template:**

```
private dataType fieldName;  
public dataType getFieldName() { return fieldName; }  
public void setFieldName(dataType fieldName) { this.fieldName = fieldName; }
```

13. Closing Summary

Day 3 introduced the core of object-oriented programming in Java: classes and objects, constructors, the `this` keyword, static members, and encapsulation. Learners understood how to model real-world entities like employees and how to protect data integrity. The corporate HR Management Portal case study demonstrated how these concepts work together to create a robust, maintainable system. Recursion and two-pointer techniques (Climbing Stairs, Merge Sorted Array) further strengthened problem-solving skills.

Key Takeaways:

- A class is a blueprint; an object is an instance.
- Constructors initialize objects; can be overloaded.

- `this` resolves naming conflicts and enables chaining.
- `static` members belong to the class, shared across instances.
- Encapsulation hides data and exposes controlled access via methods.

End of Day 3